Samraddhi Singhai 2024115015

Question 2

1. System Overview

StreetRace Manager is a command-line Python system for managing underground street races, crew members, vehicles, and missions. It is built as a set of independent modules that share state through a central data store (data.py). Each module handles one domain of the system and exposes a small set of functions that can be called from the main menu or from other modules.

1.1 Module Summary

registration.py

Registers new crew members by name and role. Prevents duplicate registrations and validates that the role is one of driver, mechanic, or strategist.

Calls: data.crew_members (write)

crew.py

Manages role reassignment and skill level tracking for registered members. All crew operations require the member to exist in data.crew_members.

Calls: data.crew_members (read/write)

inventory.py

Tracks cars, spare parts, tools, and cash balance. Exposes helpers to add/deduct stock and to flag cars as damaged or available.

Calls: data.inventory (read/write)

race.py

Creates races and enters drivers. Enforces that only registered drivers with undamaged cars may race. Uses a private _find_race helper imported by results.py to avoid duplication.

Calls: data.races (read/write), inventory.get_available_cars

results.py

Records race outcomes, marks races as completed, awards prize money to inventory, and optionally flags cars as damaged. Imports _find_race from race.py.

Calls: data.results (write), inventory.update_cash, inventory.mark_car_damaged

mission.py

Assigns missions of any type and verifies that all required crew roles are present before the mission can start.

Calls: crew.get_members_by_role

leaderboard.py [extra module 1]

Tallies wins and earnings from data.results and includes every registered crew member even if they have zero wins. Returns a sorted list of (name, stats) tuples.

Calls: data.results (read), data.crew_members (read)

garage.py [extra module 2]

Handles car repair. Enforces that only a mechanic may repair, deducts 2 spare parts per repair via inventory.deduct_parts, and clears the damaged flag on the car.

Calls: crew role check via data.crew_members, inventory.deduct_parts, data.inventory (write)

1.2 Extra Module Descriptions

leaderboard.py

The Leaderboard module provides an all-time ranking of crew members by race wins and total earnings. It reads from data.results to tally wins and prize money, then loops over data.crew_members a second time to ensure every registered member appears on the board even if they have never won a race. The final list is sorted by wins descending, then earnings descending as a tiebreaker. This module depends on both the Results module (to populate data.results) and the Registration module (to populate data.crew_members), but nothing depends on it, so it can be added without creating circular imports.

garage.py

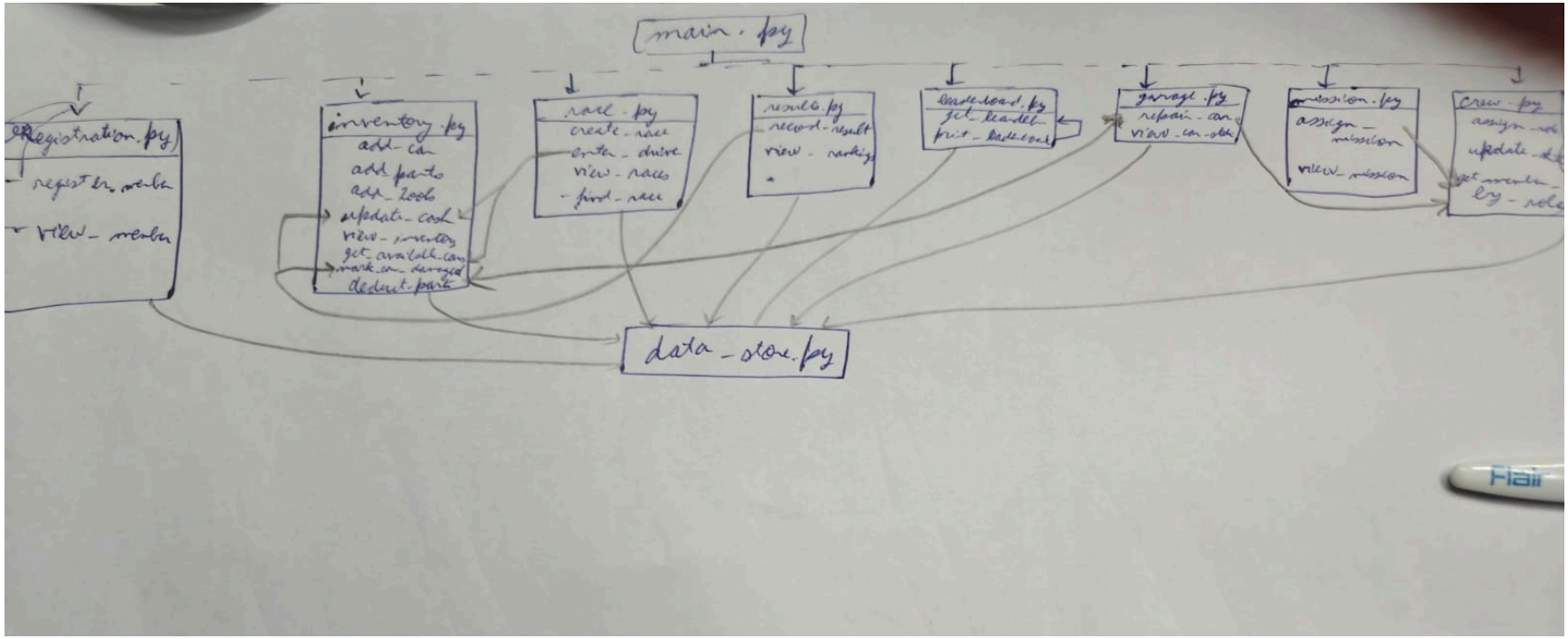
The Garage module manages car repairs after race damage. It enforces two business rules: only a crew member with the mechanic role may perform a repair, and each repair consumes exactly 2 spare parts from inventory. It checks the mechanic role directly in data.crew_members, calls inventory.deduct_parts to consume stock (which also guards against going negative), and then clears the damaged flag on the car in data.inventory. This module bridges the crew and inventory domains and is the only way to return a damaged car to a raceable state.

2. Call Graph

The call graph below shows every function in the system and the arrows between them. Each node is a single function. Grey arrows show calls within the same module. Red/orange arrows show cross-module calls, which represent the integration points tested in Section 3.

Cross-module calls:

- race.enter_driver → inventory.get_available_cars
- results.record_result → inventory.update_cash
- results.record_result → inventory.mark_car_damaged
- mission.assign_mission → crew.get_members_by_role
- garage.repair_car → inventory.deduct_parts
- garage.repair_car → crew role check (data.crew_members)
- leaderboard.get_leaderboard → data.results and data.crew_members
- leaderboard.print_leaderboard → leaderboard.get_leaderboard


main.py
registration.py
request_new_member
view_member
inventory.py
add_car
add_parts
add_tools
update_cost
view_inventory
get_available_cars
mark_car_damaged
deduct_parts
race.py
create_race
enter_drive
view_races
find_race
results.py
record_result
view_rankings
leaderboard.py
get_leaders
print_leaderboard
garage.py
repair_car
view_car_stats
mission.py
assign_mission
view_mission
crew.py
data_store.py

3. Integration Test Design

Tests are written in integration/tests/int_test.py and are grouped by the number of modules involved in each interaction. Each test calls reset() first to clear all shared state in data.py, ensuring tests are independent. The run_test() helper compares the actual return value or state to the expected value and prints PASS or FAIL.

3.1 Two-Module Tests

These tests verify that a single module correctly reads from or writes to the shared data store, or that two adjacent modules hand off data correctly.

Scenario	Modules Involved	Expected Result	Actual Result	Errors / Notes
assign_role fails for unregistered member	registration → crew	Error: 'Ghost' not registered.	PASS	None
assign_role succeeds after registration	registration → crew	Letty's role updated to mechanic.	PASS	None
update_cash adds to balance	inventory → data	cash == 1500	PASS	None
deduct_parts fails with no stock	inventory → data	Error: not enough spare parts.	PASS	None
add then deduct parts gives correct balance	inventory → data	spare_parts == 6	PASS	None
view_car_status reflects repaired state	garage → inventory	car['damaged'] == False	PASS	None

Why these tests are needed:

- The crew module must depend on registration — if assign_role could run without a registered member, role data would be inconsistent from the start.
- Cash and parts are the financial backbone of the system. If update_cash overwrites instead of adding, every race result would corrupt the balance.
- deduct_parts must guard against negative stock — without this check, a repair could proceed with zero parts and leave the system in an impossible state.
- view_car_status is the only display function in garage.py that reads inventory directly. Confirming the underlying flag is correct verifies that function would show accurate information.

3.2 Three-Module Tests

These tests verify that data flows correctly across a chain of three modules, including the key business rules around role enforcement and mission validation.

Scenario	Modules Involved	Expected Result	Actual Result	Errors / Notes
Mechanic cannot enter race	registration → crew → race	Error: 'Tej' is not a driver.	PASS	None
Driver enters race successfully	registration → inventory → race	Dom entered 'Toretto Cup' driving Charger.	PASS	None
Prize added to cash after race	race → results → inventory	cash == 4000	PASS	None
Mission blocked — missing role	registration → crew → mission	Error: mission cannot start. Missing roles: ['mechanic'].	PASS	None
Mission succeeds — all roles present	registration → crew → mission	Mission 'Nitro Heist' (heist) assigned.	PASS	None
update_skill stored correctly	registration → crew → data	crew_members['Dom']['skill_level'] == 8	PASS	None
Zero-win member appears on leaderboard	registration → leaderboard → data	'Roman' in leaderboard names	PASS	None

Why these tests are needed:

- The driver-only race rule spans three modules. If race.py does not check data.crew_members, a mechanic could race and the results module would award prize money incorrectly.
- The prize money flow (race → results → inventory) is the most important business rule. A bug here would silently lose money every race.
- Mission role validation is the only guard preventing under-staffed missions from starting. Testing both the fail and pass case confirms the check is not too strict or too lenient.
- The update_skill test was added specifically because no other test exercised the crew → data write path. An error was found here: the test originally used the key 'skill' but data.py stores it as 'skill_level'. Fixed by reading the actual data.py schema.

3.3 Four-Module Tests

These tests verify longer chains involving the damage and repair cycle, which connects race results, inventory state, and garage operations.

Scenario	Modules Involved	Expected Result	Actual Result	Errors / Notes
Car flagged damaged after race	registration → race → results → inventory	damaged_cars[0]['name'] == 'RX-7'	PASS	None
Damaged car blocked from next race	race → results → inventory → race	Error: car 'RX-7' is not available.	PASS	None
Repaired car available again	registration → inventory → garage → inventory	'Mustang' in available_cars	PASS	None

Repair deducts 2 spare parts	registration → inventory → garage → inventory	spare_parts == 3	PASS	None
Repair fails with no parts	registration → inventory → garage → inventory	Error and car still damaged	PASS	None

Why these tests are needed:

- The damage flag set by results.py is the only thing that blocks a car from the next race. If mark_car_damaged does not write correctly, damaged cars would continue racing with no consequence.
- Testing that a damaged car is blocked in the next race confirms that race.py reads the live inventory state, not a snapshot taken at entry time.
- The repair cycle test (restore → available) confirms that garage.py correctly writes False to car['damaged'] and that inventory.get_available_cars reads that updated value.
- The no-parts repair test is a negative case that confirms garage.py aborts cleanly. Without this guard, a repair would clear the damage flag even when there are no spare parts to justify it.

3.4 Five-Module Tests

These tests simulate complete real-world workflows spanning five or more modules to verify end-to-end data integrity.

Scenario	Modules Involved	Expected Result	Actual Result	Errors / Notes
Full cycle: race → damage → repair → re-enter	registration → race → results → inventory → garage → race	Dom entered 'Race 2' driving Charger.	PASS	None
Leaderboard ranks by wins after multiple races	registration → race → results → inventory → leaderboard	board[0][0] == 'Dom'	PASS	None
Car damage does not affect mission crew check	registration → race → results → inventory → mission → crew	Mission 'Parts Run' assigned.	PASS	None
Cash accumulates correctly across 3 races	registration → inventory → race → results → inventory	cash == 11500	PASS	None

Why these tests are needed:

- The full damage-repair-reenter cycle is the most complex workflow in the system. A single broken link in any of the six modules involved would cause a silent failure that would be hard to trace without an end-to-end test.
- The leaderboard ranking test confirms that results from multiple races accumulate correctly and that the sort order is maintained. An error was found here: the test used

board[0]['name'] but get_leaderboard() returns tuples, not dicts. Fixed by reading the leaderboard.py source and changing to board[0][0].

- The mission-after-damage test verifies that car state and crew state are stored independently in data.py. If they were coupled, a damaged car would incorrectly block a mission that only needs a mechanic, not the car itself.
- The three-race cash accumulation test is the definitive proof that update_cash is additive. It would catch a reset bug (cash going back to 1000 each race) or a multiplication bug immediately.